

GPU-Accelerated Reinforcement Learning and Graph-Based Network Encoding for MATSim Large-Scale Charging Infrastructure Simulations

Track 6

Abstract—Existing electric vehicle charging station placement works in MATSim have utilized external reward servers that serialize simulation outputs each iteration, creating large IO and data transfer overheads. This work presents a CUDA-enabled pipeline that keeps MATSim in-process via JPytype, packages dependencies inside a shaded uber.jar, feeds simulator state to a GraphGPS encoder that emits fixed-size tensors and instruments a read-only TensorBoard stack with reproducible manifests. The modified architecture significantly decreases socket overhead and memory footprint, making large experiments practical on consumer-grade systems.

Keywords— *Electric Vehicle Charging, MATSim, GPU acceleration, Reinforcement Learning*

I. INTRODUCTION

In “A Framework for Joint Wireless and Plug-In Charger Placement with Reinforcement Learning in MATSim,” [5] serialized each iteration into files, posted them to a reward server, and parsed the response before policy updates. This led to massive memory footprints and limitations on maximum network size that hinder practical scalability.

The goals of this work are to re-design the simulation architecture to make large scale experiments practical by: (i) keeping MATSim [2] in-process, sharing memory and a failure domain with proximal policy optimization (PPO), (ii) encoding road networks with a graph-aware transformer to capture spatial structures with fewer iterations [7], (iii) ensuring the MATSim charging subsystem operates deterministically with parallelized agents and rapid PPO policy mutations, and (iv) providing reproducible and comprehensive run manifests and diagnostics.

A. Contributions:

- Embedded MATSim: A JPytype [3] launcher plus a shaded jar allow PPO and MATSim to run in the same process, replacing the legacy HTTP reward server.
- GraphGPS observation encoder: Fixed-K line-graph sampling combined with GIN and multi-head attention produces AMP-friendly tensors [1], [7].
- GPU-resident PPO: Patched Stable-Baselines3 [6] buffers, automatic mixed precision (AMP) and CUDA graphs – multi-discrete actions remain on-device [4].
- Single Environment Multithreading: Charging logic for dynamic, static, and plug-in EV chargers modified to log agent charge queuing and charging behaviors globally, enabling robust multithreading.
- Telemetry and manifests: The simulation operates at 3 second intervals, rather than 1 second. A read-only

TensorBoard callback and per-run manifest records seeds, jar checksum, hyperparameters, etc.

II. SYSTEM DESIGN

A. Shaded Packaging and Sandboxed Runs

We build a shaded .jar with a pinned dependency graph. Each run clones the MATSim scenario into a sandbox directory, rewrites absolute paths to local ones, and deletes the sandbox afterward. This prevents cross-run contamination [2].

B. Unified Launch Workflow

A Python CLI constructs Gymnasium environments [8], sets the CUDA device, loads configuration, and seeds Python, NumPy, PyTorch, Gymnasium, and JPytype [3], [4], [8]. It writes args.json and manifest.json to output_dir/<timestamp>/. The launcher detects stale sandboxes and checkpoints to avoid reuse and provides a single entry point for both local and cluster runs.

C. MATSim In-Process via JPytype

In Java, the launcher wires MATSim [2] modules, applies queue-handling fixes, and attaches an in-memory RewardProbe. PPO calls a run_iteration() method once per rollout step; Java computes the next charging behavior and returns only scalar reward components and lightweight telemetry [3].

III. GRAPH-AWARE OBSERVATION ENCODER

The observation encoder is based on GraphGPS and is designed to produce fixed-size CUDA tensors that remain stable across rollouts and environments [7], [1], [4].

A. Pipeline:

- Top-K sampling: We select up to K nodes from a line-graph representation of the network, using degree-based ranking. This ensures fixed tensor shapes for Stable-Baselines3 [6] and focuses on high-throughput corridors where congestion and charging constraints are most important.
- Local and global mixing: Stacked GIN layers perform local mixing, while multi-head self-attention provide global context [1], [7]. LayerNorm and residual connections improve numerical stability, especially for automatic mixed precision.
- Pooling and features: Mean and max pooling generate a fixed-dimensional embedding. Charger statistics are concatenated, and scalar context is appended.
- Automatic mixed precision (AMP): The encoder uses torch.autocast, if NaNs are detected in a step, fp32 is

used for that step only, preserving stability without sacrificing training throughput in other steps [4].

- CUDA graph capture: Encoder forward and backward passes are captured in CUDA graphs, significantly reducing kernel launch overhead and iteration jitter [4].

IV. PPO TRAINING STACK AND DIAGNOSTICS

A. Policy Loop & Control Flow

The proximal policy optimization (PPO) [6] loop is structured around GPU residency and minimal host-device thrash:

- Observation: GraphGPS encodes MATSim state into a fixed-size tensor on GPU [2], [7].
- Action sampling: MultiDiscrete logits are sampled on CUDA; each dimension maps to (none, static, dynamic) for a charger or link.
- Apply actions: Actions are converted to Java enums through JPytype [3], mutate the sandbox configuration, and trigger one MATSim iteration [2].
- Reward Assembly: The RewardProbe aggregates energy delivered, queue penalties, CAPEX terms, and other signals. Reward and telemetry are returned to Python as compact scalars and short vectors, without large serialization overhead.
- Buffer add (GPU): A patched Stable-Baselines3 buffer keeps actions, observations, and rewards on GPU until training. Host-device transfers are limited to what is necessary for logging [6].
- Policy updates: after num_steps transitions, PPO updates on GPU with AMP [4], [6]. Post update, TensorBoard logs, reinforcement learning (RL) metrics, reward decomposition, domain statistics, and runtime statistics. Best policies trigger a best_chargers.csv export.

B. GPU Residency and Stability

A custom buffer patch keeps transitions on CUDA, eliminating repeated host-device copies [4], [6]. CUDA graphs capture forward and backward passes per step, smoothing kernel launch jitter and improving wall-clock performance. Charging logic improvements (e.g. listener fallbacks, dynamic charger concurrency) help ensure rollouts do not crash under aggressive policy reassignments or rapidly changing charger configurations.

C. Telemetry and TensorBoard

- RL metrics: policy loss, value loss, entropy, KL divergence, clip fraction, explained variance.
- Reward decomposition: energy delivered, queue penalties, CAPEX penalties, and the aggregated scalar reward
- Domain statistics: charger occupancy histograms, queue-length statistics, number of upgrades and downgrades per-rollout.
- Runtime statistics: MATSim step time, encoder time, PPO update time, GPU memory usage.

D. Reproducibility & Resilience

A single seed is propagated to Python, NumPy, PyTorch, Gymnasium, JPytype, and Java [3], [4], [8]. Each MATSim instance runs in an isolated sandbox with relative paths [2]. Checkpoints are saved at a user-defined frequency, and best-reward snapshots include the current charger placements.

V. DISCUSSION

The engineering modifications made remove the reward server and its HTTP connection layer [5], providing unified telemetry and faster iteration. Running MATSim and PPO in the same process simplifies debugging, reduces configuration drift, and significantly decreases compute resource usage [2], [3]. CUDA utilization and graph capture allow training and inference to be practical on mid-range consumer hardware [4]. In testing, each agent added to the simulation increases system RAM usage by ~250kb, allowing for simulations of tens of thousands of agents. The primary limitation of this framework is the static top-K sampling. Demand-weighted or congestion aware sampling, or adding the number of K nodes to use as part of the PPO logic is left as potential future work [7].

A. Conclusion

The updated pipeline converts the existing configuration of using MATSim with RL and PPO for Electric Vehicle charger placement from a socket-bound, memory-intensive workflow into an embedded, GPU-first system with graph-native observations, deterministic telemetry, and hardened charging logic. On a GTX 1650 Ti and a Ryzen 5 4600, the entire Wasatch front road network (highways, primary, secondary roads), with one agent, 1024 timesteps, a K of 1024, and a transformer embedded dimension of 384 completed in 22 hours, with about 20 spent on transformer training. During this run, it consumed approximately 2.6GB of VRAM and 1.5GB of system memory. The companion study [5] uses the legacy tool for charger placement studies, while this study documents the architecture that enables larger agent counts, broader geographic scope, and more efficient PPO updates. Final metrics for detailed comparison of compute time, resource usage, and model performance will be reported and integrated following the completion of an 8192 timestep run with 16384 agents, to more accurately represent traffic flows over the span of an entire day.

REFERENCES

- [1] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in Proc. ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
- [2] A. Horni, K. Nagel, and K. W. Axhausen, Multi-Agent Transport Simulation MATSim. London, U.K.: Ubiquity Press, 2016.
- [3] JPytype Contributors, "JPytype User Guide," available online: <https://jpytype.readthedocs.io/>, accessed: 11/1, 2025.
- [4] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2019.
- [5] I. Peterson and M. Harper, "A Framework for Joint Wireless and Plug-In Charger Placement with Reinforcement Learning in MATSim," unpublished.
- [6] A. Raffin et al., "Stable-Baselines3: Reliable Reinforcement Learning Implementations," Journal of Machine Learning Research, vol. 22, no. 268, pp. 1–8, 2021.
- [7] O. Rampásek et al., "Recipe for a general, powerful, scalable graph transformer," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2022.

- [8] M. Towers et al., “Gymnasium: A Standard Interface for Reinforcement Learning Environments,” 2024.